# DB Structure

## 1 Catalog Table – stores all books

```
CREATE TABLE catalog (
    id SERIAL PRIMARY KEY,          -- unique book ID
    title VARCHAR(255) NOT NULL,    -- book title
    topic VARCHAR(100) NOT NULL,    -- book topic (distributed systems / undergrad school)
    quantity INT NOT NULL DEFAULT 0,-- number of books in stock
    price NUMERIC(10,2) NOT NULL    -- book price
);
```

### Example Data

```
INSERT INTO catalog (title, topic, quantity, price) VALUES
('How to get a good grade in DOS in 40 minutes a day', 'distributed systems', 5, 40),
('RPCs for Noobs', 'distributed systems', 5, 35),
('Xen and the Art of Surviving Undergraduate School', 'undergraduate school', 5, 45),
('Cooking for the Impatient Undergrad', 'undergraduate school', 5, 30);
```

---

## 2 Orders Table – stores all purchase orders

```
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,                -- unique order ID
    book_id INT NOT NULL REFERENCES catalog(id) ON DELETE CASCADE,  -- reference to catalog
    title VARCHAR(255) NOT NULL,          -- book title for logging
    purchase_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP -- timestamp of purchase
);
```

### Example Data

```
INSERT INTO orders (book_id, title) VALUES
(2, 'RPCs for Noobs');  -- example order
```

---

## 3 Optional: Users Table – if you want to add users later

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100),
    password_hash VARCHAR(255)
);
```

---

## 4 Optional: Order Items Table – if you want multi-book orders in one transaction

```
CREATE TABLE order_items (
    id SERIAL PRIMARY KEY,
    order_id INT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    book_id INT NOT NULL REFERENCES catalog(id),
    quantity INT NOT NULL DEFAULT 1
);
```

---

## 5 Recommended Indexes

```
-- Speed up search by topic
CREATE INDEX idx_catalog_topic ON catalog(topic);

-- Speed up queries by purchase date
CREATE INDEX idx_orders_date ON orders(purchase_date);
```

---

## 6 Example Transaction for a Purchase

```
BEGIN;

-- Lock the book row to avoid race conditions
SELECT quantity FROM catalog WHERE id = 2 FOR UPDATE;

-- Check stock
-- If quantity > 0, decrement stock
UPDATE catalog SET quantity = quantity - 1 WHERE id = 2;

-- Log the order
INSERT INTO orders (book_id, title) VALUES (2, 'RPCs for Noobs');

COMMIT;
```

.

---

**Summary**:

- **catalog** → all books + stock + price
- **orders** → all purchases with timestamp
- **users** → optional, for future extension
- **order_items** → optional, for multi-book orders
- All Microservices (Frontend / Catalog / Order) interact via REST API.